SupportSense

Presentation Script

What to say for each slide, and which code to show if asked

Aman · 230065 **Shashwat** · 230127
Natural Language Processing

Part	Slides	Speaker	Time
1. Introduction	1–6	Aman → Shashwat	2.5 min
2. Dataset	7–10	Aman	2.5 min
3. Preparing the text	11–14	Shashwat	2.5 min
4. Category model	15–19	Aman	3 min
5. Sentiment and urgency	20–23	Shashwat	2.5 min
6. Live demo	24 + app	Aman	3 min
7. Limitations and conclusion	25–28	Shashwat	2.5 min
Total			**≈ 18.5 min**

How to use this: don’t read it word for word. Learn the main points of each slide and say them in your own words. Point at the screen while you talk.

Part 1 · Introduction

Goal: explain what SupportSense is, why it is needed, and what it does.

Slide 1 · Title

Aman · 15 s

ON SCREEN

Project name and both of our names.

SAY

Good morning. I am Aman, 230065, and this is Shashwat, 230127.
Our NLP project is **SupportSense**, a system that reads customer support tickets and sorts them automatically.

Slide 2 · Every ticket is read by hand

Aman · 35 s

ON SCREEN

Three problems on the left. An inbox on the right where a hacked account is 4th in line.

SAY

Companies get hundreds or thousands of support messages every day. Today, a person reads each one and decides where it goes.
This has three problems. It is **slow**. It is **inconsistent**: two people may sort the same message differently. And **urgent problems get buried**. In this inbox, a hacked account is waiting behind three simple questions.

Slide 3 · Our solution

Aman · 25 s

ON SCREEN

Blue slide with three questions: what, how the customer feels, how urgent.

SAY

SupportSense reads every ticket as soon as it arrives and answers three questions: **What is it about? How does the customer feel? How urgent is it?**
It is **not a chatbot**. It does not reply to customers. It helps the support team decide what to handle first.
Shashwat will show you an example.

Slide 4 · One message in, three answers out

Shashwat · 30 s

ON SCREEN

One real message on the left, three results on the right.

SAY

Here is a real result from our system. The message is: *“My account was hacked and someone changed my email.”*

Category: **Account**, 90% confident. Sentiment: **Negative**. Urgency: **High**, because it found the word “hacked”.

So this ticket goes straight to the top of the queue.

Slide 5 · Three features, three techniques

Shashwat · 35 s

ON SCREEN

Three cards: category, sentiment, urgency.

SAY

Each answer uses a different method.

Category is our machine learning model: TF-IDF plus Logistic Regression, trained on about 19,000 messages.

Sentiment uses VADER, a ready-made model, because our data has no sentiment labels.

Urgency uses simple rules: keywords like “hacked” or “charged twice”, plus the sentiment score.

Everything is shown in a Streamlit dashboard, with the most urgent tickets first.

Slide 6 · What we’ll cover

Shashwat · 15 s

ON SCREEN

Six numbered topics.

SAY

Next we will cover the dataset, how we cleaned the text, the model and its results, sentiment and urgency, a live demo, and finally the limitations.

Part 2 · Dataset

Goal: which data we used, why, what it looks like, and how it shaped our design.

Slide 7 · We compared two public datasets

Aman · 40 s

ON SCREEN

Table comparing Bitext and Tobi-Bueck, and the “data breach labelled medium” example.

SAY

We needed customer messages with correct labels. We compared two free datasets from Hugging Face.

The **Tobi-Bueck** dataset has priority labels, but more than half of it is in German and the labels are messy. A quick test model got only **52%** and **62%**. For example, “Urgent help needed for possible data breach” is labelled *medium* priority.

The **Bitext** dataset is in English, has no missing values, and the same test got **99.7%**. So we chose Bitext.

Slide 8 · What the data looks like

Aman · 35 s

ON SCREEN

Five real rows (message → category) and four key numbers.

SAY

Bitext has **26,872 messages**. We use two columns: the **message**, which is the input, and the **category**, which is what we predict.

Messages are short, about 9 words. There are no missing values, but there are 2,237 duplicates, which we remove.

The data has real-looking mistakes: typos like “rtying”, slang like “wanna”, and placeholders like {{Order Number}}.

Slide 9 · 11 categories, not balanced

Aman · 40 s

ON SCREEN

Bar chart of the 11 categories; Account and Cancel highlighted.

SAY

There are 11 categories, but they are not the same size. Account has almost 6,000 messages; Cancel has only 950, about **six times fewer**.

To handle this we did three things: a **stratified split**, **balanced class weights**, and we report **macro F1**, which treats every category equally.

Note: in this data, Cancel means cancellation *fees*. Cancelling an order is in Order.

Slide 10 · The labels decided our design

Aman · 35 s

ON SCREEN

Dark slide: category is labelled, sentiment and urgency are not.

SAY

Bitext has category labels, but **no sentiment and no urgency labels**. Without labels we cannot train a model.

So category uses a **trained model**, sentiment uses the ready-made **VADER**, and urgency uses **rules**.

One honest point: Bitext is made from templates, so it is cleaner than real tickets. That is why our accuracy is so high.

Part 3 · Preparing the text

Goal: show that the data was prepared carefully and that we avoided data leakage.

Slide 11 · Light cleaning, on purpose

Shashwat · 35 s

ON SCREEN

Four cleaning steps and two messages before and after.

SAY

Before training we clean the text, but only a little. Four steps: make it **lowercase**, remove **placeholders** like {{Order Number}}, remove **numbers** like #12588, and remove **extra spaces**.

For example, “find my invoice #12588” becomes “find my invoice”.

This is one function, **clean_text()**. The dashboard uses the same function for new tickets.

Slide 12 · What we deliberately keep

Shashwat · 35 s

ON SCREEN

“I can pay ≠ I can’t pay” and three things we chose not to do.

SAY

What we **don’t** remove is just as important.

We keep **stop words**, because they include “not” and “can’t”. “I can pay” and “I can’t pay” mean opposite things. With stop words removed, both become just “pay”.

We keep **typos** because real customers make them. And we skip **stemming** because short messages don’t need it.

Slide 13 · Removing duplicates prevents leakage

Shashwat · 40 s

ON SCREEN

Row counts from 26,872 down to 23,776, and why duplicates matter.

SAY

We checked the data step by step. No empty messages and no message with two labels. But there were **3,096 duplicates**, which we removed. That leaves **23,776** messages.

Why? If the same sentence is in both training and testing, the model is tested on something it already memorised. The score looks better than it really is. This is called **data leakage**.

Slide 14 · A stratified 80/20 split

Shashwat · 40 s

ON SCREEN

Train/test bar, a table of category shares, and two rules.

SAY

We split the data: **80% for training**, 19,020 messages, and **20% for testing**, 4,756 messages the model never sees while learning.

The split is **stratified**: each category has the same share in both parts. We fix the random seed at **42**, so the split is the same every time.

And TF-IDF learns only from the training data, which is another rule against leakage.

Part 4 · Category model and results

Goal: how TF-IDF and Logistic Regression work, how well the model does, and how it handles doubt.

Slide 15 · TF-IDF turns words into numbers

Aman · 40 s

ON SCREEN

IDF of “the” vs “refund”, and the word weights for one real message.

SAY

A model needs numbers, not words. **TF-IDF** gives each word a number. The number is high when a word is **rare** across all messages.

“The” is everywhere, so it scores low: 2.4. “Refund” is rare, so it scores high: 4.9.

We also use word pairs like “money back”. In total we have **4,823 features**. Words never seen in training, like “headphones”, get zero.

Slide 16 · Logistic Regression picks the category

Aman · 40 s

ON SCREEN

Probability bars (Refund 90.9%) and the top words per category.

SAY

Logistic Regression learns a weight for every word for every category. For a new message it adds up the weights and turns them into **probabilities** that add up to 100%.

For our example it says **Refund, 90.9%**.

We can also see what it learned: Refund is linked to “refund”, “reimbursement” and “money”. That makes sense.

Slide 17 · Results on 4,756 unseen messages

Aman · 35 s

ON SCREEN

99.6% accuracy, 0.997 macro F1, 17 mistakes; comparison with SVM.

SAY

We tested on 4,756 messages the model had never seen. **Accuracy is 99.6%**, macro F1 is 0.997, and there are only **17 mistakes**.

We also tried a **Linear SVM**. It was slightly better, 99.8%, but it does not give probabilities.

We need probabilities for our confidence check, so we kept Logistic Regression.

Slide 18 · Where it goes wrong

Aman · 35 s

ON SCREEN

Confusion matrix and four real mistakes.

SAY

This is the **confusion matrix**. Rows are the true category, columns are the prediction.
Almost everything is on the diagonal, which means correct.
Most mistakes are **heavy typos** like “paymdnt” or “invooce”.
Honestly, the score is this high because the data is clean. Real tickets would score lower.

Slide 19 · Knowing when it’s unsure

Aman · 35 s

ON SCREEN

The 50% rule, the “charged twice” example, and 99.9% vs 82.4%.

SAY

So we added a safety check. If the model is **less than 50% sure**, the ticket is marked **“Needs manual review”**.
For example, “I was charged twice” gets only 23%, because these words are rare in our data.
In the test set, 74 tickets were flagged. Confident tickets are **99.9%** correct; flagged ones only **82.4%**. So the flag catches the risky ones.

Part 5 · Sentiment and urgency

Goal: how VADER scores tone, how the urgency rules work, and how well they do, honestly.

Slide 20 · Sentiment with VADER

Shashwat · 40 s

ON SCREEN

“This is good” → “not good” → “NOT good!!!” with scores, and the score scale.

SAY

For sentiment we use **VADER**. It is a ready-made tool with a list of about 7,500 scored words, plus rules for how people write.

“This is good” scores +0.44. Add “not” and it becomes −0.34. Write NOT in capitals with “!!!” and it gets stronger: −0.51.

Below −0.05 we call it **Negative**; above 0.5, **Positive**. We use 0.5 because polite words like “please” make normal requests look happy.

Slide 21 · Urgency: five rules, checked in order

Shashwat · 35 s

ON SCREEN

Five rules, each with a real example and result.

SAY

Urgency uses **five rules**, checked in order. The first rule that matches wins.

One: security, money, legal or “urgent” words → **High**. Two: a very angry tone → **High**. Three: problem words like “can’t” or “error” → **Medium**. Four: any negative tone → **Medium**. Five: everything else → **Low**.

Every result shows its **reason**.

Slide 22 · Why keywords and tone together

Shashwat · 35 s

ON SCREEN

A calm but urgent message, and an angry message with no keyword.

SAY

Why use both? Each one alone misses something.

“I need access to my account urgently” sounds calm, but the word “urgently” makes it High.

“Absolutely terrible, I am furious” has no keyword, but its very negative tone makes it High.

Together, they catch both.

Slide 23 · How well it works, honestly

Shashwat · 40 s

ON SCREEN

94%, 100%, 100% High recall; the warning; two known failures.

SAY

We had no labels, so we wrote and labelled **36 test messages** ourselves. Sentiment was **94%** correct, urgency **100%**, and **no urgent ticket was missed**.

But we wrote these messages ourselves, so the numbers are **optimistic**.

We know where it fails: “I was not charged twice” is still High, and sarcasm like “Great, another late delivery” is scored positive.

Part 6 · Live demo

Goal: prove the system works end to end. Aman drives the laptop. Start the app before the viva.

Slide 24 · Let’s see it running

Aman · 3 min

ON SCREEN

Demo slide for a moment, then switch to the browser.

SAY

Now let’s see it running. We’ll analyse three tickets and then look at the dashboard.

Step	Do this	You will see	Say
1	Click **Hacked account** → Analyze ticket	Account 90% · High (security: hacked) · Negative	The category is Account, urgency High, and here is the reason: the word “hacked”.
2	Click **Question** → Analyze ticket	Payment 97% · Low · Neutral	A simple question gets Low urgency. It can wait.
3	Click **Double charge** → Analyze ticket	Feedback 27% · yellow “Needs manual review” · High (money: charged twice)	Urgency is correctly High. But the model is only 27% sure of the category, so it asks a human to check.
4	Empty the text box → Analyze ticket	“Please enter at least 2 words.”	Empty input doesn’t crash the app.
5	Scroll down to **Ticket dashboard**	Total, High priority, % negative, needs review; three charts	Every ticket is saved here. A team lead sees the totals and charts at a glance.
6	In **All tickets**: click the **High** filter, then the review toggle, then search “refund”	Table with High tickets first, then filtered results	The queue puts urgent tickets first. Agents can filter and search. That’s the full system. Shashwat will now talk about the limitations.

Avoid: the *Late delivery* example gets the wrong category (it is flagged for review). If it comes up, say: “Wrong category, but low confidence, so it’s sent to a human.”

Part 7 · Limitations and conclusion

Goal: show we know the weak points, have a plan, and can sum up in one line.

Slide 25 · What it can’t do (yet)

Shashwat · 50 s

ON SCREEN

Six limitation cards, each with a real example.

SAY

Here is what our system can’t do yet.

One, the biggest: the training data is **synthetic**, so real accuracy would be lower. Two: only **one category** per ticket. Three: our sentiment and urgency test set is **small and written by us**. Four: keyword rules don’t understand “**not**”. Five: VADER misses **sarcasm**. Six: **English only**.

Slide 26 · What we’d do next

Shashwat · 40 s

ON SCREEN

Three columns: data, models, system.

SAY

Each limitation points to a next step.

Data: use real company tickets and the priorities agents give. **Models:** train a real urgency model, try DistilBERT if the data is messy, allow several categories, and handle “not”. **System:** use a database, send tickets to the right team automatically, and connect to email.

Slide 27 · Conclusion

Shashwat · 35 s

ON SCREEN

One-line takeaway and three numbers.

SAY

To sum up: SupportSense reads a message, predicts its category, sentiment and urgency, and puts urgent tickets first.

99.6% category accuracy, **no urgent ticket missed** in our tests, and **zero crashes** in 18 edge-case tests.

Our main point: simple, explainable NLP can sort support tickets, and it **knows when to ask a human**.

Slide 28 · Thank you

Both · 5 s

ON SCREEN

Thank-you slide with both names.

SAY

Thank you. We are happy to take your questions.

Appendix · If they ask to see the code

Open the SupportSense folder in your editor and zoom in. In VS Code, press Ctrl+G and type a line number to jump to it.

Start here: `src/analyzer.py`. Its function `analyze_ticket()` is the whole system in 20 lines: it checks the input, runs the three features, and returns one result.

Code tour: if they say “walk us through the code”

#	Open (file : lines)	Point at	Say
1	`src/analyzer.py : 14–33`	`analyze_ticket()`	The heart of the system. It calls our three features and returns one result.
2	`src/preprocessing.py : 9–18`	`the three re.sub lines`	Our cleaning: lowercase, remove placeholders, numbers and extra spaces. No stop-word removal.
3	`notebooks/03_train_model.py : 45–59`	`TfidfVectorizer, Pipeline, model.fit`	TF-IDF with word pairs, then Logistic Regression. One Pipeline, so TF-IDF learns from training data only.
4	`notebooks/03_train_model.py : 126`	`joblib.dump`	The trained model is saved as one file, which the app loads.
5	`src/predict.py : 13, 25–35`	`CONFIDENCE_THRESHOLD, predict_proba`	Get probabilities, take the highest, flag for review if below 50%.
6	`src/urgency.py : 41–65`	`# Rule 1 … # Rule 5`	Five rules in order. The first match wins, always with a reason.
7	`src/sentiment.py : 15–28`	`the two thresholds`	VADER gives a score; these thresholds turn it into a label.
8	`streamlit_app.py : 70–85`	`the form, analyze_ticket, save_tickets`	Clicking Analyze calls the same function and saves the ticket to a CSV file.

When asked a specific question

If they ask…	Open	Show
Where do you clean the text?	`src/preprocessing.py`	lines 9–18
How did you remove duplicates and split the data?	`notebooks/02_prepare_data.py`	line 39 (drop_duplicates); lines 43–45 (train_test_split, stratify, random_state=42)
Where is TF-IDF? What settings?	`notebooks/03_train_model.py`	lines 45–51
Where is the model trained?	`notebooks/03_train_model.py`	lines 53–59
How did you evaluate it?	`notebooks/03_train_model.py`	line 66 (classification_report); image reports/confusion_matrix.png
How did you compare with SVM?	`notebooks/03_train_model.py`	line 71
How does the confidence check work?	`src/predict.py`	lines 13 and 25–35
Show the urgency keywords	`src/urgency.py`	lines 10–29 (HIGH_KEYWORDS, MEDIUM_KEYWORDS)

If they ask…	Open	Show
How is urgency decided?	`src/urgency.py`	lines 41–65
How is sentiment done?	`src/sentiment.py`	lines 15–28
What happens with empty input?	`src/analyzer.py`	lines 11–17 (MIN_WORDS, ValueError)
Where are tickets stored?	`src/storage.py`	lines 16–35, and the file data/tickets.csv
Show the dashboard code	`streamlit_app.py`	lines 70–85 (input form); line 117 onward (dashboard)
How do you know it doesn't crash?	`notebooks/05_final_tests.py`	the cases list; run it live

Run it live in the terminal (from the SupportSense folder)

To do this	Type this
Start the dashboard	`.venv/bin/streamlit run streamlit_app.py`
Analyse any message	`.venv/bin/python -c "from src.analyzer import analyze_ticket; print(analyze_ticket('My account was hacked'))"`
Re-run training and evaluation (about 10 s)	`.venv/bin/python notebooks/03_train_model.py`
Re-run the edge-case tests	`.venv/bin/python notebooks/05_final_tests.py`

Numbers to remember

What	Number
Dataset	Bitext, 26,872 messages, 11 categories
After removing duplicates	23,776 (3,096 removed)
Train / test	19,020 / 4,756 (80/20, stratified, seed 42)
TF-IDF features	4,823 (words + word pairs)
Category accuracy / macro F1	99.6% / 0.997 · 17 mistakes
Flagged for review (below 50%)	74 tickets (1.6%) · 99.9% vs 82.4% correct
Sentiment / urgency (36 messages)	94% / 100% · High recall 100%
Edge-case tests	0 crashes in 18